11/6/2026

IMDb Sentiment Classification with TF IDF and SVM

Natural Language Processing course project for the Master of Data Science

GROUP NO.03

Students name and ID:

NGUYEN DOAN ANH HUY – 20261210M

NGUYEN PHUC HUNG – 20261205M

DAO ANH QUAN – 20261211M

NATURAL LANGUAGE PROCESSING COURSE FINAL BIG PROJECT

IMDb Sentiment Classification with TF IDF and SVM

Comparing linear and RBF classifiers for movie reviews

Programmed: Master of Data Science
Course and instructor: Natural Language Process – Dr. Nguyen Kiem Hieu

This report presents our NLP course project. Which will represent what our group have achieved, done and experiments during the whole course.

Abstract

In this project, we use TF-IDF features and support vector machines to classify English movie reviews as positive or negative. We focus on a few practical questions from our NLP course: how much word order helps, whether an RBF kernel improves on a linear kernel, and whether cleaning text or removing common words makes a useful difference. We chose this setup because it lets us examine these choices through controlled comparisons.

Our main experiment uses 20,000 training reviews and a new balanced holdout of 6,000 IMDb reviews. We compare six text representations with linear and RBF SVC. Three-fold cross-validation gives 216 fits across 72 parameter configurations, followed by 12 final refits. The vocabulary and IDF are fitted separately inside each training fold. Cross-validation selects a cleaned unigram/bigram representation with RBF SVC, C = 2, and gamma = scale.

The selected model achieves 89.300% accuracy and 89.385% positive-class F1 on the holdout. Adding bigrams to the raw RBF representation improves F1 by 0.913 percentage points, while adding trigrams gives only another 0.005 points. The reported paired bootstrap intervals include zero for the matched RBF-versus-linear bigram comparison, cleaning, and common-word removal. We therefore treat those smaller differences cautiously.

Our earlier CPU experiment adds a useful view of training cost. LinearSVC reaches 89.500% accuracy with an 8.9-second pipeline fit, compared with 89.517% and 1,540.0 seconds for RBF SVC. That study uses a different holdout and tuning design, so we keep its scores separate from the GPU results. The CPU subset ablations also find no significant accuracy difference from keeping all words when stop words are removed, including with negation and contrast retained. Small linguistic tests also reveal errors with longer negation, contrast, and reported speech. The main lesson is that a strong review-level score can coexist with clear weaknesses on individual sentences.

Keywords

Sentiment analysis, IMDb, TF-IDF, n-grams, SVM, RBF kernel, cross-validation, ablation, reproducibility.

Contents

1 Research problem and scope 4

2 Text representation 5

3 Classifiers and evaluation measures 7

4 Data selection and holdout design 8

5 Model selection protocol 9

6 Experimental configurations 10

7 Main holdout results 11

8 Effect of n gram length 12

9 Uncertainty and kernel comparison 13

10 Common word and stop word ablations 14

11 Supporting CPU experiment 16

12 Controls and result verification 17

13 Linguistic diagnostics 18

14 Demonstration and local execution 19

15 Discussion and conclusions 20

References 21

Sections 4–9 describe the main GPU study and its feature and kernel comparisons. Section 10 covers common-word removal and the separate CPU stop-word ablation. Section 11 compares CPU training and prediction time. The later sections discuss controls, language tests, and the demonstration.

1 Research problem and scope

Motivation

A movie review can contain more than one opinion. Consider the illustrative sentence “The acting was excellent, but I would not recommend the film.” The acting receives praise, but the overall judgment is negative. Our task assigns one positive or negative label to the whole review. This example explains the task rather than reporting a model prediction. Negation and contrasting views make the problem especially interesting for an NLP course.

We use TF-IDF and SVM as a manageable baseline for studying this problem. We can change the word features, kernel, regularization, and vocabulary size without changing the entire modeling approach. This makes it easier to connect a result to an experimental choice. We also include targeted sentence tests because the average IMDb score does not tell us whether the classifier handles a phrase such as “not good” or a change of opinion later in a sentence.

Research questions

Question	Experimental evidence
Do bigrams or trigrams help?	Unigram, unigram/bigram, and up-to-trigram comparisons under a 12,000-feature budget.
Does RBF improve enough to justify its cost?	Same-backend linear/RBF comparisons, paired F1 intervals, and separate CPU cost evidence.
Should common words and preprocessing change?	HTML/contraction cleaning and two controlled movie/film feature-removal variants.
What does strong IMDb performance miss?	Locked diagnostic sentences and pair-level correctness.

Scope and contribution

Our task is limited to the overall positive or negative sentiment of English movie reviews. We do not model neutral opinions, sentiment toward individual aspects, or Vietnamese text. The project includes the comparison code, saved settings and results, control experiments, and a demonstration workflow that supports CPU inference. Prediction margins are shown as decision values rather than confidence percentages.

We report the earlier CPU experiment and the later GPU experiment separately. They use different holdouts, sampling procedures, tuning scopes, implementations, and timing definitions. Keeping these differences visible helps us avoid attributing a score change to the hardware alone. The local notebook adapts the later workflow for rerunning on a personal computer, but it does not yet provide a completed local IMDb result.

2 Text representation

TF IDF and normalization

TF-IDF converts each review into a sparse numerical vector. Most entries are zero because a review contains only a small part of the vocabulary. Term frequency captures how often a term appears in that review, while inverse document frequency reduces the relative weight of terms found in many training reviews. Our vectorizer uses smoothed IDF, sublinear term frequency, and L2 normalization [2].

For N training documents, the smoothed formula is idf(t) = ln((1 + N) / (1 + df(t))) + 1, where df(t) counts documents containing term t. Sublinear TF replaces a nonzero count tf with 1 + ln(tf). We then normalize each weighted review vector to unit L2 length. The vocabulary and document frequencies must come from the relevant training partition, including during cross-validation.

An important detail is the +1 in the IDF formula. Even a word appearing in every training document has a nonzero IDF. A common word can therefore still influence the classifier. This gives us a reason to test words such as movie and film rather than assume that TF-IDF automatically makes them irrelevant.

A small TF IDF calculation

For an illustrative corpus of four reviews, suppose movie appears in all four and excellent appears in two. Their smoothed IDF values are ln(5/5) + 1 = 1 and ln(5/3) + 1 ≈ 1.511. If each word appears once in a review, its sublinear TF is 1, giving weights of 1 and 1.511 before L2 normalization. The common word still has a nonzero weight. These numbers explain the formula and are not measurements from the IMDb experiment.

Word n grams

Representation	Illustrative features	Purpose
Unigram	not, good, movie	Lexical baseline
Unigram and bigram	not good, good movie	Short local combinations
Up to trigram	not good movie	Additional local ordering

The main raw representations all use max_features = 12,000 and min_df = 2. With this cap, introducing bigrams or trigrams can replace some unigrams in the vocabulary. Our comparison therefore measures the performance of each complete representation under the same feature budget. It does not isolate longer context while holding every original feature fixed.

Word order in an example

The illustrative phrases “good acting but bad story” and “bad acting but good story” contain the same unigrams. A unigram representation cannot distinguish how the praise and criticism attach to acting and story. Bigrams retain combinations such as “good acting” and “bad story”, which differ between the phrases. They capture this local order without representing the full sentence structure.

We also test a cleaned representation. It unescapes HTML, removes tags, and expands selected negative contractions before vectorization. These changes can make the input more consistent, but they do not teach the model how far negation extends or which clause expresses the reviewer’s overall view. We evaluate cleaning as a separate choice rather than assume that it always helps.

3 Classifiers and evaluation measures

Linear and RBF SVM

A linear SVM separates the classes using a weighted combination of the TF-IDF features. An RBF SVM uses the nonlinear similarity $K(x, z) = \exp(-\text{gamma} \times \|x - z\|^2)$. The parameter C controls the penalty for training errors, while gamma controls how local the RBF similarity is [3, 4]. Comparing the two kernels lets us ask whether the extra flexibility improves this sparse text baseline.

In the main study, both kernels use cuML SVC, so the backend and solver family are shared. The earlier CPU study also includes scikit-learn LinearSVC. Although its decision boundary is linear, LinearSVC uses a different implementation and default loss from SVC with a linear kernel. We keep their names separate throughout the report because their training times and results cannot be treated as interchangeable [3, 4].

Reported measures

Measure	Definition	Interpretation
Accuracy	(TP + TN) / total	Overall proportion correct
Positive precision	TP / (TP + FP)	Reliability of positive predictions
Positive recall	TP / (TP + FN)	Coverage of truly positive reviews
Positive F1	2 × precision × recall / (precision + recall)	Selection score combining positive precision and recall
Macro F1	Mean of positive and negative F1	Equal class-level contribution

The main holdout contains 3,000 reviews from each class. We select parameters by positive-class F1 and also report accuracy and macro-F1. When comparing percentages, we use percentage points: moving from 88.420% to 89.333% F1 is an increase of 0.913 points. This is different from a 0.913% relative improvement.

Training scores tell us how well the model fits reviews it has already seen. Validation and holdout scores are more useful for judging performance on new reviews. The demonstration also returns a decision margin, but we do not interpret this value as a probability that a prediction is correct.

Reading precision and recall in our result

For the selected model, 2,703 truly positive reviews receive a positive prediction and 297 receive a negative one. Positive recall is therefore 2,703 / 3,000 = 90.100%. There are also 345 false positives, so precision is 2,703 / (2,703 + 345) = 88.681%. Recall asks how many positive reviews we find, while precision asks how many of our positive predictions are correct.

4 Data selection and holdout design

IMDb source and balanced samples

We use the Large Movie Review Dataset introduced by Maas et al., loaded as stanfordnlp/imdb. The official labeled training and test splits each contain 25,000 reviews [1]. For the main experiment, we sample 10,000 training reviews per class and 3,000 holdout reviews per class, using seed 42.

We record the sampling settings and dataset revision so that the source can be identified when repeating the experiment. The reported comparisons use the same selected reviews within the main study.

New holdout and duplicate handling

To create a new holdout, the main workflow reconstructs the 6,000 official-test IDs used by the earlier sampling procedure and excludes them from the available pool. It also normalizes HTML, whitespace, and case before computing text hashes. Duplicate reviews are removed within pools, and test candidates matching a review in the official training split are excluded.

These steps reduce exact text overlap and avoid deliberately reusing the earlier test sample. There is still a limitation: the CPU results archive does not contain a dataset revision or a manifest of the original review IDs. Reconstructing those IDs depends on the assumed source order and sampling procedure. We therefore describe the exclusion as a reconstruction rather than a fully verified record of the historical sample.

An example of duplicate filtering

For example, “Great film!
” and “great film!” become the same text after removing the HTML tag and normalizing case and whitespace. A hash can identify this exact normalized match. It would not identify “I thought the film was great!” as the same review. This illustration shows why exact-text filtering reduces overlap without detecting every paraphrase.

Partition	Reviews	Class balance	Role
Main training sample	20,000	10,000 per class	Fit representations and select parameters
Main holdout	6,000	3,000 per class	Evaluate predefined comparisons
Diagnostic suite	144	Template dependent	Inspect targeted behaviors separately

We use the diagnostic sentences for a different purpose from the IMDb holdout. They are controlled examples of particular language patterns, not a representative sample of movie reviews. Their results can reveal weaknesses that deserve investigation, but they cannot estimate how often those weaknesses occur across the full dataset.

5 Model selection protocol

Fold local feature fitting

We use stratified three-fold cross-validation with shuffling and a fixed seed. Inside each fold, the vectorizer learns its vocabulary and IDF from the training rows only. It then transforms the validation rows without refitting. We cache these matrices so that kernel candidates use exactly the same representation within a fold.

This detail matters even though TF-IDF does not use the sentiment labels. Fitting it on all reviews before cross-validation would still let validation text influence the vocabulary and weights. Fitting inside each fold avoids that source of leakage. It does not address every form of dependence, such as reviews about the same film or written by the same author.

An example inside one CV fold

Suppose the word spaceship appears only in the validation part of a fold. A vectorizer fitted on that fold's training reviews will not include it in the vocabulary, so it contributes no feature when the validation review is transformed. Fitting the vocabulary on all reviews would let validation text affect the representation. This is an illustrative case, not an observed token audit.

Parameter grid and selection

Kernel	C grid	Gamma grid	Candidates per representation
Linear SVC	0.5, 2, 8	Not applicable	3
RBF SVC	0.5, 2, 8	scale, 0.1, 1	9

For each of the six representations, we test three linear configurations and nine RBF configurations. This gives 72 configurations and 216 fits over three folds. We retain the highest mean positive F1 in each of the 12 representation/kernel groups, then select the overall winner by CV. Ties are resolved deterministically using smaller C, followed by the gamma string.

We save the CV-selected parameters before evaluating the holdout. This separates model selection from final evaluation and gives us a fixed set of models to compare.

Final evaluation

We refit each selected configuration on all 20,000 training reviews and transform the holdout and diagnostic inputs with that fitted representation. The notebook saves predictions, margins, metrics, and model bundles. We keep the CV-selected winner even if another model looks attractive in a particular holdout comparison. The paired comparisons help us interpret differences; we do not use them as an additional selection step.

6 Experimental configurations

We compare six text representations with both linear and RBF SVC. The raw unigram/bigram representation serves as our reference for the ablations. Each variant changes a specific part of the representation while sharing the same review samples and cross-validation folds. This keeps the comparisons easier to interpret.

Representation ID	Change from baseline	Question
raw_w1	Unigram only	Value of short word sequences
raw_w2	Unigram and bigram	Reference representation
raw_w3	Unigram through trigram	Additional value of trigrams
clean_w2	HTML and contraction cleaning	Value of the cleaning package
raw_w2_drop_unigram	Remove movie and film unigram columns	Effect of the two isolated features
raw_w2_drop_containing	Remove all retained n-grams containing either word	Effect of the broader feature family

Meaning of feature removal

We fit the baseline vectorizer before removing feature columns. For illustration, consider movie, film, good, bad, “good movie”, and “bad film”. Removing the two unigrams keeps both phrases. Removing all features containing movie or film also removes the phrases, leaving good and bad in this small example. We filter columns rather than delete words from the sentence and join their neighbors. After filtering, we normalize the vectors again and do not add replacement features. The retained counts below refer to the actual fitted vocabulary.

Removing the two unigrams leaves 11,998 features. Removing all containing n-grams leaves 11,792, so that version removes 208 features. We tune C and gamma separately for each representation/kernel group. As a result, the final score difference compares two tuned pipelines, rather than measuring what happens when we remove a feature from one fixed fitted model.

Options not executed

The completed run does not execute the optional 30,000-feature vocabulary variant or the max-document-frequency ablation. We leave those as possible follow-up experiments. Cleaning, both common-word ablations, shuffled-label controls, and the near-duplicate audit are enabled in the recorded configuration, so these are the checks discussed in the results.

7 Main holdout results

Cross-validation selects clean_w2_rbf, which combines HTML and contraction cleaning with unigram/bigram TF-IDF and RBF SVC. Its settings are C = 2 and gamma = scale. Mean CV positive F1 is 89.535%, with a fold standard deviation of 0.393 percentage points. On the 6,000-review holdout, it reaches 89.300% accuracy, 88.681% precision, 90.100% recall, 89.385% positive F1, and 89.299% macro-F1.

Representation	Linear F1 %	RBF F1 %
Unigram	88.214	88.420
Unigram and bigram	89.017	89.333
Up to trigram	89.171	89.338
Cleaned unigram and bigram	89.101	89.385

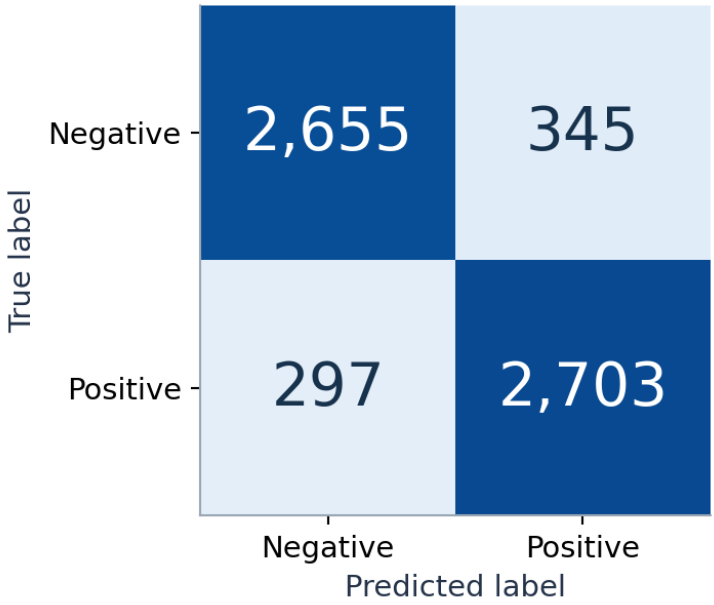


Figure 1. Champion confusion matrix reconstructed from the saved aggregate metrics. Rows are true labels.

In more concrete terms, the model gets 5,358 reviews right and 642 wrong. The reconstructed counts are TN = 2,655, FP = 345, FN = 297, and TP = 2,703. It predicts positive for more negative reviews than it predicts negative for positive reviews, which explains why positive recall is higher than precision. These counts agree with the aggregate metrics.

We use the cleaned RBF model because it won the CV selection. Its holdout lead over raw bigram RBF is only 0.052 F1 points, however, so that ranking alone gives us little evidence that cleaning makes a dependable difference. Section 9 examines the uncertainty around these small gaps.

8 Effect of n gram length

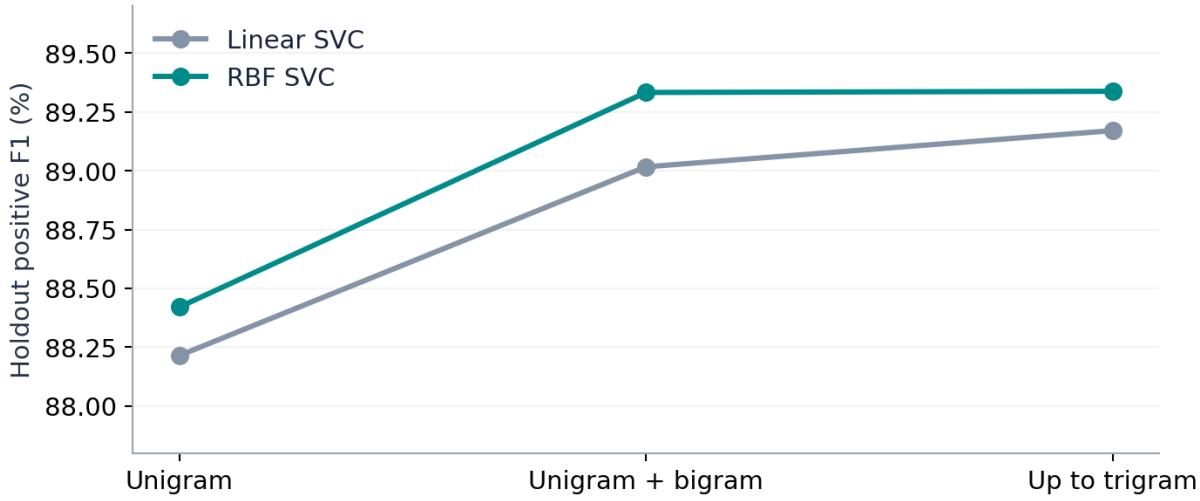


Figure 2. Main-run positive F1 under a fixed maximum vocabulary of 12,000 features. The vertical axis is truncated to expose small differences.

Bigrams give the clearest improvement in this comparison. RBF positive F1 increases from 88.420% with unigrams to 89.333% with unigrams and bigrams, a gain of 0.913 percentage points. The linear model also improves, from 88.214% to 89.017%, or 0.803 points. Because these models use the same reviews and folds, the results support including short word combinations in this setup.

The paired bootstrap table reports unigram RBF minus bigram RBF as −0.913 points, with a 95% interval of about [−1.525, −0.338]. Reversing the comparison gives a bigram gain of +0.913 points with interval [+0.338, +1.525]. This interval excludes zero for the fitted models on this holdout, giving us stronger evidence for the bigram change than for most other changes we tested.

Trigram provides little extra benefit

Adding trigrams produces a much smaller difference. Up-to-trigram RBF reaches 89.338% F1, only 0.005 points above raw bigram RBF. The reported interval runs from approximately −0.321 to +0.330 points, so we cannot conclude that trigrams improve performance consistently. Linear trigram is 0.154 points above linear bigram, but the supplied bootstrap table compares both with raw bigram RBF and does not directly test that linear-versus-linear difference.

We also need to remember the 12,000-feature limit. Longer n-grams compete with unigrams for the available vocabulary slots. These results describe the representations under that limit. Testing a larger vocabulary, character n-grams, or an uncapped vocabulary would help us investigate further, but those experiments have not been completed here.

9 Uncertainty and kernel comparison

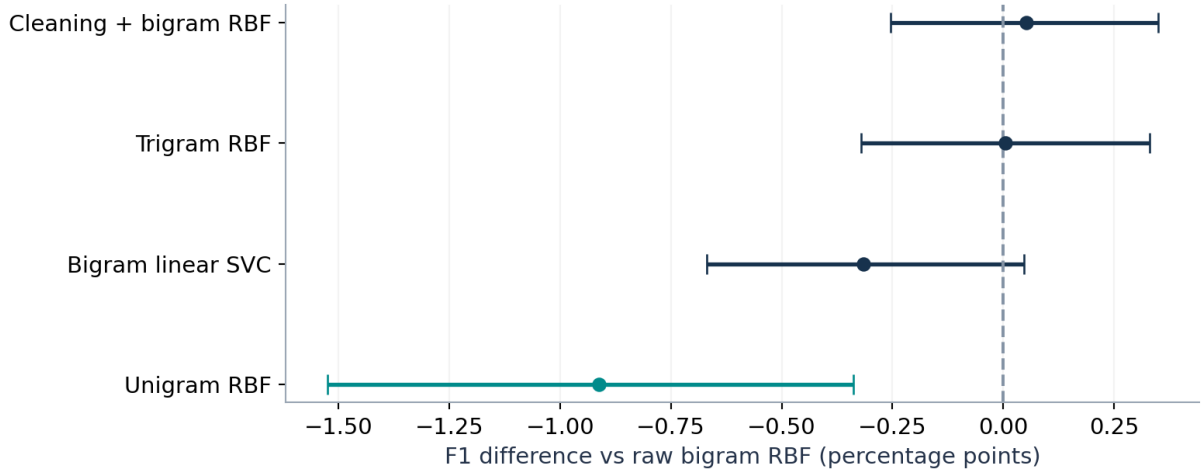


Figure 3. Reported paired-bootstrap 95% intervals. Reference is raw_w2_rbf. Differences are in percentage points.

Raw bigram RBF has a 0.316-point positive-F1 lead over raw bigram linear SVC. The table expresses the comparison in the opposite direction, giving linear minus RBF an interval of [−0.669, +0.046] points. For RBF minus linear, the result is +0.316 points with interval [−0.046, +0.669]. Since this range includes zero, we cannot claim a clear RBF advantage from this run.

Cleaning shows the same issue on a smaller scale. It adds 0.052 RBF F1 points, with a reported interval of [−0.254, +0.349]. A model can come first in the observed ranking even when the difference from another model is uncertain. For our interpretation, choosing a model by CV and showing that one preprocessing step improves performance are separate questions.

Interpretation of intervals

For each bootstrap replicate, the notebook samples the same holdout reviews for both models, stratified by class, and calculates the difference in positive F1. Using paired samples keeps the comparison aligned on the same examples. The full run uses 1,000 replicates and a fixed random seed.

These intervals capture uncertainty from sampling holdout reviews for models that are already fitted. They do not cover retraining on another sample, changing the seed, repeating parameter selection, or moving to another dataset. We also explore several comparisons without adjusting for multiple comparisons. We treat the intervals as exploratory evidence within this study.

The point differences agree with subtraction of the final F1 scores. The interval endpoints come from the original experiment; recalculating them requires the paired predictions for individual reviews.

10 Common word and stop word ablations

We test movie and film because both are frequent without being automatically irrelevant to TF-IDF. In the executed notebook, they occur in 12,279 and 11,129 of the 20,000 training reviews, with IDF values of about 1.488 and 1.586. The linear baseline gives movie a coefficient of −0.386 and film +0.360. These signs describe associations in the training data; they do not mean that either word has a fixed sentiment of its own.

Variant	RBF F1 %	RBF delta pp	Linear F1 %	Linear delta pp
Keep baseline features	89.333	0.000	89.017	0.000
Remove two unigrams	89.315	−0.018	88.962	−0.055
Remove containing n grams	89.145	−0.187	89.159	+0.142

No consistent removal benefit

Removing only the two unigrams slightly reduces F1 for both kernels. Removing all n-grams containing either word reduces RBF F1 but raises linear F1 numerically. All four reported 95% intervals contain zero. We therefore find no consistent evidence that removing these features improves the classifier.

The broader ablation removes 208 features, including 206 bigrams, so we cannot attribute its result to the two isolated words. Renormalizing also changes the relative weights of the remaining coordinates. We interpret these results as comparisons between the full filtering pipelines.

The earlier CPU study uses a different intervention. It removes movie, movies, film, and films through the vectorizer’s stop-word setting and rebuilds the representation. That can change which n-grams form and which terms enter the limited vocabulary. The CPU and GPU ablations address a similar question, but they do not repeat the same experiment.

Answer to the preprocessing question

For this dataset and setup, we would keep the reference vocabulary. Frequent words still have nonzero weights, but that alone is not a reason to remove them. The measured changes are small and uncertain. A different dataset could give another answer, which is why we would evaluate removal again rather than turn it into a general preprocessing rule.

Stop word ablation in the CPU study

We also test the stop-word question raised during the earlier presentation. The CPU experiment compares keeping all words, removing the default English stop-word list, and removing that list while preserving selected negation and contrast words. These ablations train on the 5,000-review subset and evaluate on the same 6,000-review CPU test sample. They are separate from the 20,000-review final fits and from the main GPU study.

The protected set contains no, not, nor, never, none, nobody, nothing, neither, nowhere, cannot, but, however, although, though, and yet. The custom stop-word list is the default list minus this set. For example, default removal turns the illustrative input “not good” into “good”, whereas the custom list keeps “not good”. Removing tokens before vectorization can also change which words become adjacent in a bigram. The vectorizer is refitted for each variant, with a 12,000-feature cap. The classifiers reuse the selected CPU parameters without retuning for each ablation.

Model	Stop-word setting	Accuracy %	Positive F1 %	Fit s
LinearSVC	Keep all words	87.067	87.178	2.508
RBF SVC	Keep all words	87.200	87.335	38.610
LinearSVC	Default English removal	86.933	87.063	1.955
RBF SVC	Default English removal	86.900	87.047	18.815
LinearSVC	Removal with negation and contrast retained	87.117	87.221	1.720
RBF SVC	Removal with negation and contrast retained	87.183	87.300	18.672

Default stop-word removal lowers accuracy relative to keeping all words by 0.133 percentage points for LinearSVC and 0.300 points for RBF. Preserving negation and contrast brings the results closer to the baseline: LinearSVC is 0.050 points above it and RBF is 0.017 points below it. The saved paired accuracy tests against each model’s keep-all baseline do not find significant differences. They do not directly test the custom list against the default list, so we treat that comparison as descriptive.

The removal variants fit faster in this subset experiment, but the results do not establish that stop-word removal improves accuracy reliably. We would keep the no-removal baseline and use a negation-preserving list if testing removal further. Keeping negation tokens also leaves the harder question of negation scope unresolved.

11 Supporting CPU experiment

The earlier CPU experiment compares DummyClassifier, LinearSVC, linear SVC, and RBF SVC with a shared unigram/bigram TF-IDF pipeline. In full mode, final models train on 20,000 reviews and are tested on 6,000. Tuning and preprocessing ablations use a smaller 5,000-review training subset. The archived CSV files let us compare accuracy and fitting cost within that experiment.

Model	Accuracy %	Positive F1 %	Pipeline fit s	Pipeline predict s
Dummy	49.450	49.509	8.8	1.134
LinearSVC	89.500	89.552	8.9	1.108
SVC linear	89.233	89.347	329.8	85.711
SVC RBF	89.517	89.577	1,540.0	122.083

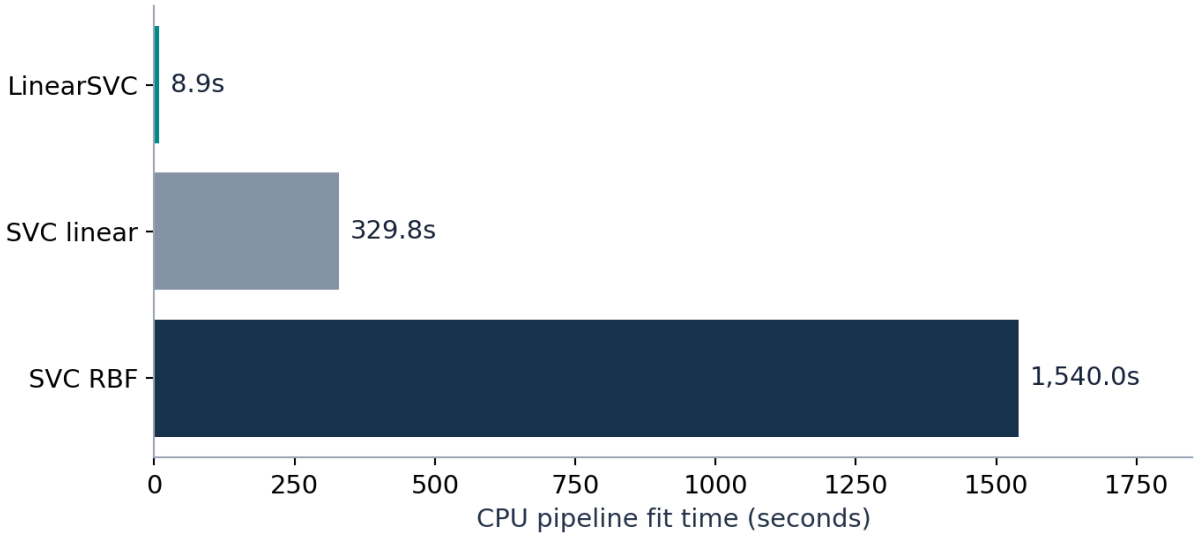


Figure 4. Earlier CPU pipeline-fit times, including TF-IDF. The main GPU table instead reports SVM fit time separately.

RBF leads LinearSVC by just 0.017 accuracy points, while its pipeline fit takes about 173 times longer. Predicting the 6,000-review test batch takes 1.108 seconds for LinearSVC and 122.083 seconds for RBF, including the pipeline transformation. The saved paired accuracy tests do not find a significant difference between RBF and either linear comparator. LinearSVC is therefore a useful CPU baseline. It uses a different implementation and default loss from SVC with a linear kernel. These measurements compare models within the CPU experiment and do not measure a GPU-versus-CPU speedup.

The earlier RBF tuning table contains a tie at gamma = 1 for C = 8 and C = 32. The final chosen parameter dictionary was not retained with the summarized results, so we do not assign a specific final setting to that historical comparison.

12 Controls and result verification

Consistency checks

Recalculating the 216 fold results reproduces the 72 candidate summaries, including F1, fold standard deviation, accuracy, macro-F1, training F1, fit-time sums, and fold counts. Applying the selection rule also reproduces the 12 selected configurations and the overall CV winner.

The final positive-F1 values agree with the recorded precision and recall, and the ablation and bootstrap point differences agree with subtraction of the corresponding final F1 values. These checks help us identify calculation or aggregation errors. They do not replace a complete rerun from the raw reviews.

Control outcomes

Control or check	Observed result	Interpretation
Shuffled-label linear accuracy	49.925%	Near chance after training-label permutation
Shuffled-label RBF accuracy	50.045%	Near chance after training-label permutation
Recorded CV workload	108 fits per GPU	Balanced recorded workload across two devices
Champion training fold F1	99.9825%	Large fit-to-validation gap
Champion CV minus holdout F1	0.150 pp	Similar aggregate CV and holdout performance

The notebook also reports no nearest-training candidates above cosine similarity 0.98, with a maximum near 0.8928. This check looks for highly similar reviews beyond the exact-text filtering. Its result is recorded from the experiment rather than recalculated here.

The shuffled-label results are close to chance, as we would expect after breaking the connection between reviews and labels. Together with the duplicate checks, they address specific concerns about the workflow. They do not rule out every possible leakage or bias. The selected model also has a training-to-CV F1 gap of about 10.45 points, which shows that its near-perfect training fit should not be confused with generalization.

13 Linguistic diagnostics

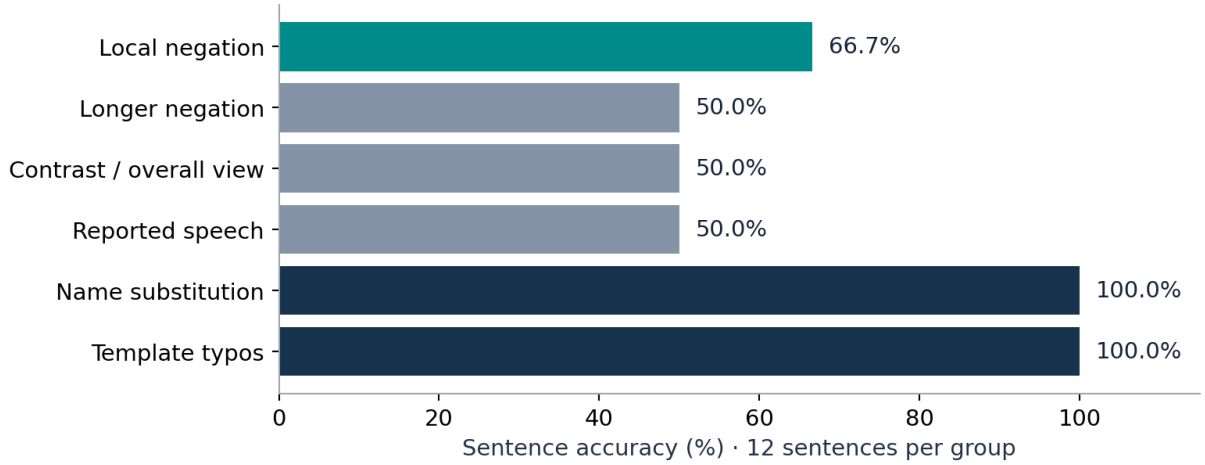


Figure 5. Champion performance on locked template groups. Each group contains only 12 sentences and 6 pairs.

The diagnostic sentences make some of the model's limitations easier to see. The selected model gets 8 of 12 local-negation sentences right, but both sentences are correct in only 2 of the 6 pairs. For longer negation, explicit overall contrast, and reported speech, accuracy is 50%, with no fully correct pair in any of those groups.

Checking pairs adds information that sentence accuracy can hide. Consider the illustrative pair "I liked the film." and "I did not like the film.", with intended positive and negative labels. Predicting positive for both gives 50% sentence accuracy and no fully correct pair. These are constructed inputs, not recorded predictions for the exact sentences. We count a diagnostic pair as correct only when both predictions match the expected labels. The three difficult groups have no fully correct pairs in the locked test.

Coverage and labeling limits

Name substitution and template typos both reach 100% locked accuracy for the selected model. These are small groups with limited label coverage, so we do not take them as evidence that arbitrary names or spelling errors are harmless. The typo results also differ between the development and locked sets, which is another reason to avoid broad claims.

Manual verification of the expected diagnostic labels is unfinished. We therefore report performance against the supplied templates, rather than present these percentages as a validated estimate of the model's language ability.

The saved demonstration in Section 14 also shows an incorrect positive prediction for a review that praises the acting but rejects the film overall. This is consistent with the contrast weakness, although one sentence cannot identify the cause of an error.

14 Demonstration and local execution

Demonstration workflow

The GPU notebook includes an export path for running the selected model and raw_w2_linear baseline on a CPU. For the linear model, it stores the fitted weights and intercept together with the vectorizer and feature mask. For RBF, it uses the fitted-model conversion where supported. Probe sentences are used to compare the exported predictions with the source model before an export is accepted.

The demonstration offers a notebook widget, a command-line interface, and a Streamlit app. A user enters an English review, chooses an available model, and receives a positive or negative label with a decision margin. Small CSV batches are also supported. The saved preprocessing, vocabulary, and mask are reused at inference time, so the demonstration does not need to retrain the model or download IMDb again.

Recorded demonstration examples

Input review	Saved prediction	Interpretation
“A wonderful film with excellent acting and a moving story.”	Positive	Matches the intended positive opinion
“The acting was excellent, but overall I disliked the film.”	Positive	Misses the intended negative overall opinion

Both predictions above appear in the saved GPU demonstration output. The second review shows why positive words alone are insufficient to determine the overall opinion. These examples illustrate behavior on individual inputs and do not change the holdout metrics.

Local retraining is a separate operation

For local retraining, the local notebook replaces cuML with sparse scikit-learn SVC. It starts in smoke mode with 400 training and 200 holdout reviews so that the workflow can be checked on a small sample. Full mode restores the larger sample sizes and grid. This starts a new local experiment; it cannot resume a cuML fit already running on Kaggle.

The local adaptation passed a synthetic-data check covering fitting, saving, prediction, and ZIP creation during preparation. It has no completed IMDb outputs in the supplied notebook. We therefore do not report a local IMDb score or assume that a Windows computer will reproduce the GPU timings. For a class demonstration, inference from a verified exported bundle is the more direct option.

15 Discussion and conclusions

Findings supported by the experiments

Our main result is that this TF-IDF and SVC setup provides a useful baseline for English movie reviews. The CV-selected cleaned bigram RBF model reaches 89.300% accuracy and 89.385% positive F1 on the new holdout. Of the representation changes, adding bigrams gives the clearest improvement over unigrams. Trigrams contribute very little more under the fixed vocabulary limit.

The kernel comparison is less decisive. RBF scores higher than matched linear bigram SVC, but its reported interval includes zero. The CPU experiment also shows that a small accuracy difference can come with a large fitting cost. For us, this makes a linear baseline worth keeping in the comparison, especially when the experiment needs to run on a personal computer. The CPU stop-word tests also give no clear accuracy reason to remove stop words in this setting.

Conclusion

For our NLP coursework, the most useful lesson is learning how to connect a model score to a specific experimental question. Bigrams improve this lexical baseline, while the smaller RBF and preprocessing gains need more evidence. The sentence errors also remind us to inspect what the classifier gets wrong. A review-level accuracy near 89% gives us a useful starting point, with clear room to improve how the model handles language.

References

[1] Maas, A. L. et al. (2011). Learning Word Vectors for Sentiment Analysis. ACL HLT, 142–150. Dataset: https://ai.stanford.edu/~amaas/data/sentiment/ Paper: https://aclanthology.org/P11-1015/

[2] scikit-learn. TfidfTransformer documentation. https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfTransformer.html

[3] scikit-learn. SVC documentation. https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html

[4] scikit-learn. LinearSVC documentation. https://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html

Documentation consulted on 6 November 2026 for definitions and implementation distinctions. Installed versions from each run govern actual behavior; current documentation is not a substitute for the recorded environment.